

2026-2027

Python in de Klas

Computationeel denken - 3e jaar IW

Leer problemen ontleden en ontwerp algoritmen met Python

1

Ontleed

Lees de opgave, controleer voorbeelden en bepaal variabelen.

2

Ontwerp

Werk een helder stappenplan uit, kies passende datatypes en controlestructuren.

3

Test en debug

Zet het algoritme om in Python, test en verbeter fouten.

NAAM _____

KLAS _____

Inhoud

Inleiding.....	3
Variabelen, datatypes, invoer en uitvoer.....	5
Variabelen	6
Datatypes	7
Invoer, verwerking en uitvoer	8
De sequentie	14
Code wordt van boven naar beneden uitgevoerd.....	15
Modulo en gehele deling	16
De math-module	19
De selectie	22
Een- en tweezijdige selectie.....	23
Meervoudige selectie	29
Herhalingen	33
Begrensde herhaling	34
Patronen tekenen	40
Random-functie	44
Introductie lijsten.....	45
Uitbreiding: rij van Fibonacci	46
Itereren over een string	47
Voorwaardelijke herhaling	49
Overzicht leerdoelen	54
Computationeel denken.....	54
Python-basis	54
Controlestructuren	54
Begrippenlijst.....	54
Formularium	55

Computationeel denken betekent dat je problemen zo ontleedt en beschrijft dat een computer kan helpen om ze op te lossen. Programmeren is een belangrijk onderdeel van die aanpak. In deze cursus gebruiken we Python als taal om onze algoritmes te ontwerpen. Je leert niet alleen syntax, maar vooral hoe je van een probleem naar een eigen algoritme gaat. Je leert programmeren door zelf te voorspellen, tekenen, schrijven, uitvoeren, testen en debuggen. Dat leren programmeren doe je door zowel de theorie (in deze syllabus) als de onlineoefeningen veelvuldig in te studeren.

Er zijn heel wat verschillende programmeertalen. Volgens sommige bronnen zijn er zo'n 9000! In deze cursus gebruiken we Python, een programmeertaal ontwikkeld door de Nederlander Guido van Rossum.

HOOFDSTUK 01

Inleiding

Van probleem naar programma

BELANGRIJKE BEGRIPPEN

- computationeel denken
- algoritme
- editor
- testen en debuggen

NA DIT HOOFDSTUK KUN JE

- Ik kan uitleggen hoe programmeren past binnen computationeel denken.
- Ik kan de stappen begrijpen, ontwerpen, schrijven en testen toepassen.
- Ik kan een geschikte editor en een online oefenomgeving gebruiken.

Computationeel denken

Een computer voert instructies precies uit. Om een probleem met een computer op te lossen, moet je het probleem eerst ontleden en omzetten in een ondubbelzinnige reeks stappen.

DEFINITIE

Algoritme

Een eindige en ondubbelzinnige reeks stappen die een probleem oplost of een taak uitvoert.

Python is een programmeertaal waarin we zulke algoritmen kunnen beschrijven. De taal werd ontwikkeld door Guido van Rossum en is populair omdat de code meestal helder leesbaar is.

Oefening 1.1 · Programmeertalen herkennen

Noteer minstens drie programmeertalen die je al kent.
Beschrijf bij een ervan waar die taal voor gebruikt wordt.

TIP

Denk hiervoor terug aan eerdere programmeertalen die je zag binnen de opleiding. Vaak zijn programmeertalen ontwikkeld om een bepaald probleem aan te pakken, zoals het ontwerpen van websites of het bijhouden van een enorme database.

Oefening 1.2 · Verken de oefenomgeving (Dodona of Delphi)

Deze syllabus kan je gebruiken samen met de cursus 'Python in de Klas'. Je vindt deze cursus terug op twee digitale oefenomgevingen, namelijk Delphi of Dodona. Beide hebben hun eigenschappen, voordelen en nadelen, maar telkens gaat het over een digitale omgeving waar je oefeningen kan vinden en via de ingebouwde editor deze kan oplossen. Ga na bij de vakdocent welke omgeving gebruikt wordt en doorloop samen de interface.

[Open de Delphi-leeromgeving](#)

[Open de Dodona-leeromgeving](#)

TIP

Er is maar één manier om te leren programmeren: namelijk door het zelf te doen. Dit betekent dat je veel oefeningen moet maken. Deze bundel is handig om de syntax te leren kennen, daarnaast kunnen de voorbeelden en vragen helpen bij het nadenken over problemen. Het belangrijkste ingrediënt van het leerproces is echter zelf oefeningen maken, ongeacht de leeromgeving. Dat doe je best gespreid, veelvuldig en met een vaste studieroutine.

HOOFDSTUK 02

Variabelen, datatypes, invoer en uitvoer

Gegevens bewaren, verwerken en tonen

BELANGRIJKE BEGRIPPEN

- variabelen
- datatypes
- input()
- print()
- debuggen

NA DIT HOOFDSTUK KUN JE

- Ik kan data in duidelijke variabelen bewaren en aanpassen.
- Ik kan int, float, str en bool herkennen en omzetten.
- Ik kan een programma opbouwen met invoer, verwerking en uitvoer.
- Ik kan syntax-, runtime- en logische fouten onderscheiden.

Variabelen

DEFINITIE

Variabele

Een naam die verwijst naar een waarde in het geheugen.

Met de toekenningoperator = geef je een waarde aan een variabele.

Code 2.1 - Een waarde toekennen

```
leeftijd = 16
```

variabele	waarde
leeftijd	16

Oefening 2.1 · Variabelen lezen

Beschouw de code. Beantwoord de bijhorende vragen.

```
a = 7  
b = a
```

"a krijgt de waarde _____, b _____"

Oefening 2.2 · Een waarde aanpassen

Start met leeftijd = 16. Voer daarna leeftijd = leeftijd + 1 uit.

Leg uit welke waarde in de variabele leeftijd zit aan het einde van ons algoritme.

```
leeftijd = 16  
leeftijd = leeftijd + 1
```

LET OP

Drie soorten fouten

- Een syntaxfout schendt de taalregels.
- Een runtime-fout ontstaat tijdens het uitvoeren.
- Een logische fout laat het programma uitvoeren, maar geeft een verkeerd resultaat.

LET OP**Afspraken bij variabelen**

- Een variablenaam begint niet met een cijfer en bevat geen spaties of operatoren.
- Kies een betekenisvolle naam. Gebruik snake_case voor meerdere woorden.
- Python is hoofdlettergevoelig: leeftijd en Leeftijd zijn verschillende namen.

Oefening 2.3 · Namen en fouten debuggen

Corrigeer elk fragment. Benoem ook of het probleem een onbestaande naam, hoofdletterfout of ongeldige variablenaam is.

```
# Fragment 1
basis = 6
hoogte = 8
oppervlakte = basis * Hoogte / 2
```

```
# Fragment 2
aantal_munten = 20
bedrag = 40
1munt = bedrag / aantal_munten
```

Datatypes

type	betekenis	voorbeeld
int of integer	geheel getal	getal = 4
float	kommagetal met puntnotatie	temperatuur = 18.5
str of string	tekst tussen aanhalingstekens	spreuk = "Expecto Patronum"
bool of boolean	waar of onwaar	is_minderjarig = True

Code 2.2 - Het type kan veranderen

```
x = 2
x = x + 3.5
```

Oefening 2.4 · Datatypes herkennen

Noteer bij elke uitdrukking het datatype: str, int, float of bool.

- "Harry"
- "2 + 5.0"
- 8 / 4
- 3 < 5

Invoer, verwerking en uitvoer

Gegeven volgende code:

```
zijde = float(input("Geef de zijde in: "))  
omtrek = 3 * zijde  
print("De omtrek is", omtrek)
```

1. Vraag de invoer aan de gebruiker met een input-functie.
2. Zet de invoer om naar het juiste datatype.
3. Verwerk de waarden met een formule of algoritme.
4. Toon een duidelijke uitvoer met een print-functie.

Oefening 2.5 · Invoer, verwerking en uitvoer volgen

Gegeven volgende invoeren; welke waarde steekt op het einde in variabele 'omtrek'?

Zijde = 5, dan is de omtrek: _____

Zijde = 2.0, dan is de omtrek: _____

Omtrek = 8.4, dan is de zijde: _____

FOUTMELDING

```
zijde = float(input("Geef de zijde in: ")) ← we voeren hier 'vijf' in.  
omtrek = 3 * zijde
```

...

ValueError!

```
ValueError: could not convert string to float: 'vijf'
```

input() levert standaard tekst. float() kan alleen tekst omzetten die een geldig getal voorstelt.

Code 2.4 - Tekstinvoer

```
naam = input("Geef je naam in: " )  
print(naam)
```

Oefening 2.6 · Meerdere argumenten, machten en afronden

```
a = 2  
b = 1.2  
print("De som is", a + b)  
print("Het product van", a, "en", b, "is", a * b)  
c = round(b ** a, 1)  
print(c)
```

In bovenstaand stukje code vind je heel wat nieuwe elementen terug. Merk op hoe de `print()` functie in regel 4 twee argumenten bevat. In regel 5 bevat deze zelfs zes argumenten. Hoe werkt de komma, in deze functie?

In regel 7 zie je een nieuwe bewerking `**`. Welke bewerking stelt dit voor?

Je vindt er ook een nieuwe functie `round()`. Wat doet deze functie?

DIGITALE OEFENING**Wie is Alice**

Open jouw oefenomgeving en los volgende oefening op: 'Wie is Alice'. Noteer jouw algoritme hieronder:

Oefening 2.7 · Concateren met plus of komma

```
voornaam = "Darth"  
familienaam = "Vader"  
naam = voornaam + familienaam  
print(naam)
```

In bovenstaand stukje code concateren we met een plusteken en twee strings. Wat is het verschil tussen concateren met een plusteken versus een kommateken?

FOUTMELDING**TypeError**

```
leeftijd = 16
print("Hallo, ik ben " + leeftijd + ".")
```

De + operator kan hier geen str en int samenvoegen.

Zet leeftijd om met str() naar het datatype string of geef meerdere argumenten aan print().

Code 2.5 - Expliciete omzetting

```
leeftijd = 16
print("Hallo, ik ben " + str(leeftijd) + ".")
```

Oefening 2.8 · Meer datatypes herkennen

Bepaal het datatype van elke waarde of uitdrukking.

- "Ik voetbal." _____
- "Hallo" _____
- 7.0 _____
- "7.0" _____
- 1.2345 _____
- 7 _____
- True _____
- 8 < 5 _____

Oefening 2.9 · Toekenningen en uitvoer voorspellen

Voorspel de uitvoer van beide codefragmenten.

Noteer ook de eindwaarde van elke variabele.

```
getal = 7
getal = 9
print(getal)
```

```
S = 1000
S = S + 100
S = S + 200
S = S - 50
print(S)
```

Oefening 2.10 · Een booleaanse variabele

Welk datatype krijgt `is_negatief`?

Geef twee mogelijke invoerwaarden en de bijbehorende waarde van `is_negatief`.

```
getal = int(input("Geef een getal in: "))  
is_negatief = getal < 0  
print(is_negatief)
```

Oefening 2.11 · Gemengde fouten debuggen

Debug volgende codefragmenten:

```
tekst = 'Hallo'  
print(tekst)
```

```
tekst = "Er is een doos"  
print(text)
```

```
leeftijd = input(int("Geef je leeftijd in: "))
```

```
snelheid = float(input("Voer de snelheid in: "))
```

```
postcode = int(input("Voer je postcode in: "))  
print("Je postcode is", postcode + ".")
```

DIGITALE OEFENINGEN

Kennismaking

Open jouw oefenomgeving en los volgende oefening op:

- Rapportcijfers
- Drukkerij

Oefening 2.12 · Logische fouten herstellen

De algoritmes worden uitgevoerd, maar de formule is telkens fout.
Corrigeer de omtrek van een rechthoek en de verkoopprijs met 20 procent winst.

```
lengte = 5  
breedte = 7  
omtrek = 2 * lengte + breedte
```

```
inkoop = float(input("Geef de inkoopprijs in: "))  
verkoop = 0.2 * inkoop
```

Oefening 2.13 · Afronden voorspellen

Voorspel de uitvoer.
Let op: 7,2357 met een komma is in Python geen kommagetal met dezelfde betekenis als 7.2357. In Python schrijven we decimale getallen met een punt, niet met een komma.

```
print(round(7.2357, 3))
```

```
print(round(7.2357, 2))
```

```
print(round(7.2357))
```

```
print(round(7.9357))
```

DIGITALE OEFENINGEN

Afronden tot X Cijfers

Open jouw oefenomgeving en los volgende oefening op:

- Schrijnwerkerij
- Brandstofverbruik
- Fahrenheit
- Middencirkel

Oefening 2.14 · Oppervlakte van een trapezium

Schrijf een programma dat met minstens vier variabelen de oppervlakte van een trapezium met grote basis 7, kleine basis 4 en hoogte 3 berekent. Gebruik telkens zowel voor de invoer als voor de uitvoer een volledige zin.

DIGITALE OEFENINGEN**Concateneren van string en getal**

Open jouw oefenomgeving en los volgende oefening op:

- Online Bestelling
- Online Bestelling Deel 2
- Oppervlakte en Omtrek Rechthoek
- Oppervlakte en Omtrek Vierkant
- Stappenteller

HOOFDSTUK 03

De sequentie

Volgorde, gehele deling en de math-module

BELANGRIJKE BEGRIPPEN

- volgorde
- importeren
- // en %
- math
- afronden

NA DIT HOOFDSTUK KUN JE

- Ik kan verklaren waarom instructievolgorde belangrijk is.
- Ik kan gehele deling en modulo toepassen op ontledingsproblemen.
- Ik kan functies uit de math-bibliotheek importeren en gebruiken

Code wordt van boven naar beneden uitgevoerd

Een programma gebruikt drie fundamentele controlestructuren: sequentie, selectie en herhaling. Bij een sequentie wordt elke instructie in de geschreven volgorde uitgevoerd.

FOUTMELDING

NameError door verkeerde volgorde

```
y = x + 1  
x = 7  
print(y)
```

Wat loopt er hier verkeerd?

Herschrijf de code om de fout op te lossen:

Oefening 3.1 · Instructies ordenen

Plaats de instructies in een volgorde waardoor x op het einde 46 is.

```
y = y - 1  
y = 2 * x  
x = x + 3 * y  
x = 7
```

Oefening 3.2 · Waarden verwisselen

Start met $a = 9$ en $b = 11$. Onderzoek welke programma's ervoor zorgen dat a op het einde 11 is en b op het einde 9. Teken na elke regel de toestand van a , b en eventueel c . Schrijf daarna zelf een correcte oplossing met een tijdelijke variabele.

```
# Programma A
a = b
b = a

# Programma B
c = b
a = b
b = c

# Programma C
c = a
a = b
b = c

# Programma D
c = a
a = c
c = b
b = c
```

Programma	start a	start b	start c	eind a	eind b	eind c
A	9	11	-			-
B	9	11	leeg			
C	9	11	leeg			
D	9	11	leeg			

Modulo en gehele deling

DEFINITIE

Gehele deling

$14 // 4 = 3$: vier past drie volledige keren in veertien.

De gehele deling geeft dus het quotiënt (=geheel getal) en negeert de rest.

Modulo

$14 \% 4 = 2$: twee is de rest.

De modulo-deling geeft dus de rest (=geheel getal) en negeert het quotiënt.

Oefening 3.3 · // en % evalueren

Voorspel de uitvoer van elk fragment en leg uit welke rest of welk quotiënt je berekent. Controleer nadien jouw voorspelling met de code-editor.

```
print(27 % 7)
```

```
print(83 // 5)
```

```
print((134 % 100) // 10)
```

```
print((134 // 10) % 10)
```

Oefening 3.4 · Een getal met drie voorwaarden

Zoek het unieke natuurlijke getal x waarvoor $x // 11 = 8$, $x // 13 = 6$ en $x \% 7 = 6$. Beschrijf je zoekstrategie.

DIGITALE OEFENINGEN

Complexe operatoren

Open jouw oefenomgeving en los volgende oefening op:

- Modulo Deling
- Even en Oneven Kwadraten
- Honderdtallen

Oefening 3.5 · Een overbodige bewerking herkennen

Leg uit waarom `rest // 1` overbodig is en vereenvoudig het programma.

```
bedrag = 38
aantal_5 = bedrag // 5
rest = bedrag % 5
aantal_1 = rest // 1
print(aantal_5, "biljetten van 5 en", aantal_1, "munten van 1")
```

Oefening 3.6 · Pond omzetten naar kilogram en gram

Zet 4589 pond om naar een geheel aantal kilogram en een resterend aantal gram.
Gebruik `1 pond = 0.45359237 kg` en leg uit waar gehele deling of modulo nuttig is.

De math-module

Een bewerking die we nog niet behandeld hebben, is de worteltrekking. Die is standaard niet aanwezig in Python, maar we kunnen de bewerking eenvoudig inladen door een wiskundige module te importeren. Dit doe je zo:

```
import math    # Dit plaats je in het begin van jouw algoritme.
```

Via de math-module kunnen we een vierkantswortel trekken van een getal. De functie `math.sqrt()` neemt de (positieve) vierkantswortel (`sqrt` is de afkorting van `square root`). De variabele `getal` bevat in dit geval het kommagetal 4.0. Het resultaat van `math.sqrt()` is steeds een float!

```
import math

getal = math.sqrt(16)
print(getal)
```

Naast de functie `math.sqrt()` bevat de module `math` nog heel wat andere extra's, zoals `pi`, `floor` en `ceil`. Via de volgende oefening trachten we de toepassing van die extra's te achterhalen.

Oefening 3.7 · `floor()` en `ceil()` verklaren

Voorspel de uitvoer en leg het verschil uit tussen `floor()` en `ceil()`.

```
import math
getal = math.pi
print(getal)
print(math.floor(getal))
print(math.ceil(getal))
```

Oefening 3.8 · Error! Math-code debuggen

Corrigeer de import-, formule-, datatype- en naamfouten in de gegeven fragmenten. Noteer bij elke verbetering het soort fout.

```
# Fragment 1
import.math
print(round(math.pi, 3))
```

```
# Fragment 2
getal = math.sqrt(225)
print(getal)
```

```
# Fragment 3
zijde = 8
oppervlakte = zijde * 2
print("De oppervlakte van het vierkant is", oppervlakte)
```

```
# Fragment 4
oppervlakte = 144
zijde = oppervlakte / 2
print("De zijde van het vierkant is", zijde)
```

```
# Fragment 5
import math
getal = math.sqrt(225)
print("De vierkantswortel van 225 is", getal + ".")
```

```
# Fragment 6
getal 1 = math.sqrt(144)
print(getal 1)
```

Oefening 3.10 · Afrondingsvoorwaarden combineren

Zoek een waarde x waarvoor $\text{round}(2*x) = 8$, $\text{floor}(3*x) = 11$ en $\text{ceil}(4*x) = 16$.
Er zijn meerdere oplossingen. Leg je keuze uit.

Oefening 3.11 · Een samengestelde uitvoer

Vul het programma aan zodat een enkele `print()` de tekst 3.14159 met 2 vermenigvuldigen is 6.283 toont. Gebruik variabelen x , y en resultaat.

```
import math
x = math.pi
y = 2
resultaat = ...
```

DIGITALE OEFENINGEN

Import Math, Pi en Wortels

Open jouw oefenomgeving en los volgende oefening op:

- Straal Voetbal
- Bureau van Pythagoras
- Grenswaarden
- Behangpapier
- Koeriersbedrijf
- Ruit
- Dodentocht

HOOFDSTUK 04

De selectie

Code laten beslissen

BELANGRIJKE BEGRIPPEN

- vergelijkingen
- if en else
- booleaanse operatoren
- elif
- stroomdiagrammen

NA DIT HOOFDSTUK KUN JE

- Ik kan voorwaarden schrijven met vergelijkings- en booleaanse operatoren.
- Ik kan een- en tweezijdige selecties bouwen.
- Ik kan meerdere gevallen ordenen met elif.
- Ik kan selecties uitwerken, tekenen en debuggen.

Een- en tweezijdige selectie

Code 4.1 - Eenzijdige selectie

```
temperatuur = float(input("Geef de temperatuur in: "))
if temperatuur < 0:
    print("Het vriest.")
```

LET OP

Dubbele punt en inspringing

Een if-regel eindigt met een dubbele punt.

De code die bij de voorwaarde hoort, springt vier spaties in.

Operator	Uitleg
<	kleiner dan
<=	kleiner dan of gelijk aan
>	groter dan
>=	groter dan of gelijk aan
==	gelijk aan
!=	niet gelijk aan

BOOLEAANSE OPERATOREN

and, or en not

- **and** is alleen waar wanneer beide deelvvoorwaarden waar zijn.
- **or** is waar wanneer minstens een deelvvoorwaarde waar is.
- **not** keert True en False om.

Code 4.2 - Een booleaanse variabele als voorwaarde

```
temperatuur = float(input("Geef de temperatuur in: "))
is_vriesweer = temperatuur < 0
if is_vriesweer:
    print("Het vriest.")
```

FOUTMELDING

Toekennen is niet vergelijken

```
naam = input("Geef je naam in: ")
if naam = "Voldemort":
    print("Je naam mag niet gezegd worden.")
```

Dus: gebruik == om gelijkheid te controleren.

Code 4.3 - Tweezijdige selectie

```
temperatuur = float(input("Geef de temperatuur in: "))
if temperatuur < 0:
    print("Het vriest.")

else:
    print("Het dooit.")
```

Oefening 4.1 · Booleaanse uitdrukkingen voorspellen

Neem $a = 6$ en $b = 7$.

Voorspel de uitvoer van vergelijkingen met $==$, and , or en not .

Leg bij twee uitdrukkingen uit waarom de uitkomst $True$ of $False$ is.

```
a = 6
b = 7
```

```
print(a == 6)
```

```
print(a == 7)
```

```
print(a == 6 and b == 7)
```

```
print(a == 7 and b == 7)
```

```
print(a == 7 or b == 7)
```

```
print(not a == 6 or b == 7)
```

```
print(a == 7 or b == 6)
```

```
print(not (a == 6 or b == 7))
```


Oefening 4.2 · Selecties traceren

Bepaal per fragment welke regels worden uitgevoerd en wat uiteindelijk wordt afgedrukt. Let op fragmenten waarin de if-code niet wordt uitgevoerd.

```
# Fragment 1
leeftijd = 17
if leeftijd >= 18:
    print("Hoera, u wordt gezien als een volwassene!")
```

```
# Fragment 2
import math
gok = 3.14
if gok != math.pi:
    print("Je hebt het getal niet geraden.")
```

```
# Fragment 3
getal = 57
if getal < 58:
    b = getal % 5
print(b)
```

```
# Fragment 4
getal = 58
if getal < 58:
    b = getal % 5
else:
    b = getal // 10
print(b)
```

DIGITALE OEFENINGEN

Werken met IF + Werken met IF en ELSE

Open jouw oefenomgeving en los volgende oefening op:

- Gratis verzending
- Positief, negatief of nul

Oefening 4.3 · Voorwaarden debuggen

Zoek de eventuele fouten in de fragmenten en debug deze.

```
# Fragment 1
lengte = 1.2
if lengte > 1.0 and <= 1.9:
    print("Je mag op de attractie.")
```

```
# Fragment 2
getal = 27
if getal => 28:
    print("Het getal is groter of gelijk aan 28!")
```

```
# Fragment 3
getal = 27
if getal > 30:
    b = getal + 2
    print(b)
else getal <= 30:
    print(getal)
```

```
# Fragment 4
leeftijd = 17
if leeftijd >= 18:
    print("Hoera, u wordt gezien als een volwassene!")
else:
    print("U bent nog niet volwassen...")
```

Oefening 4.4 · Deelbaar door 3 en 4

Geef drie invoerwaarden waarvoor `getal % 4 == 0` and `getal % 3 == 0` waar is.

Oefening 4.5 · Absolute waarde

Vul een tweezijdige selectie aan die de absolute waarde van een ingevoerd getal toont in een volledige zin, zoals "De absolute waarde van ... is ..."

```
getal = int(input("Geef een getal in:"))
```

Oefening 4.6 · Driehoeksongelijkheid

De driehoeksongelijkheid zegt dat drie zijden enkel een driehoek kunnen vormen indien de som van elke twee zijden steeds groter is dan de derde zijde.

Vul het programma aan en geef op het scherm weer of de zijden al dan niet een driehoek vormen.

```
a = float(input("Geef de lengte van de eerste zijde in:"))  
b = float(input("Geef de lengte van de tweede zijde in:"))  
c = float(input("Geef de lengte van de derde zijde in:"))
```

DIGITALE OEFENINGEN

Werken met IF + Werken met IF en ELSE

Open jouw oefenomgeving en los volgende oefening op:

- Onbekende A en B
- Prijzen Taxi's

NIEUW

`min(a, b)` geeft de kleinste waarde en `max(a, b)` de grootste.

Deze functies worden in de volgende roosteropdracht gebruikt.

Meervoudige selectie

Code 4.4 - Een selectie binnen een selectie

```
a = int(input("Geef a in: "))
b = int(input("Geef b in: "))
if a == b:
    print("a en b zijn gelijk")
else:
    if a < b:
        print("a is het kleinst")
    else:
        print("b is het kleinst")
```

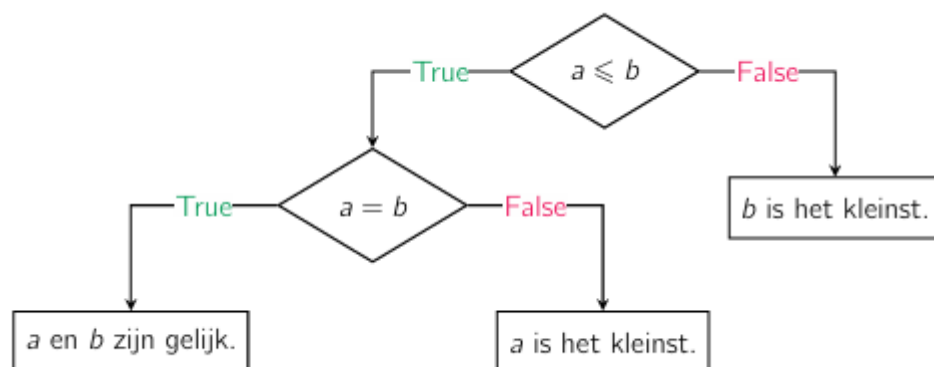
Code 4.5 - Meerdere gevallen met elif

```
temperatuur = float(input("Geef de temperatuur in: "))
if temperatuur < 0:
    print("Het vriest.")
elif temperatuur < 4:
    print("Lichte dooi.")
else:
    print("Het dooit.")
```

De volgorde van voorwaarden is belangrijk. Zodra een if- of elif-voorwaarde waar is, worden de volgende takken van dezelfde keten niet meer onderzocht.

Oefening 4.7 · Van stroomdiagram naar code

Vorm het stroomdiagram om naar een Python-programma. Schrijf eerst de geneste selectie en herwerk het programma daarna met elif.



```
a = int(input("Geef een eerste getal in: "))  
b = int(input("Geef een tweede getal in: "))
```

DIGITALE OEFENINGEN

Werken met IF, ELIF en ELSE

Open jouw oefenomgeving en los volgende oefening op:

- Gratis Verzending Deel 3
- Museumbezoek Griekenland
- Korting op Pizza
- Risk.

Oefening 4.8 · Losse if versus elif

Vergelijk code met twee losse if-opdrachten met code waarin de tweede tak elif gebruikt. Voorspel telkens de uitvoer van een codefragment.

```
# Fragment 1
getal = 101
if getal > 100:
    print("optie A")
if getal > 50:
    print("optie B")
else:
    print(getal)
```

```
# Fragment 2
getal = 101
if getal > 100:
    print("optie A")
elif getal > 50:
    print("optie B")
else:
    print(getal)
```

```
# Fragment 3
getal = 101
if getal > 101:
    print("optie A")
elif getal < 50:
    print("optie B")
else:
    print(getal)
```

Oefening 4.9 · Een complexe selectie analyseren

De stoppen zijn doorgeslagen bij een arme informaticus die de volgende code schreef.

```
v = int(input("Geef een getal in: "))

if (v > 1 and v <= 2) or (v >= 4 and v <= 6):
    if (v > 2 and v < 4) or (v > 5 and v <= 9):
        print("Luke, I am your father!")
    else:
        if v >= 3 and v <= 4:
            print("May the force be with you!")
else:
    print("Hhhrrrrrrrrrrrrraaaaaaaaaaaaaaeerrrrrrrr")
```

Bron: be-OI 2016

Zal dit dan "Hhhrrrrrrrrraaaaaaaaaaaaaaeerrrrrrr" printen?
(Als je dit niet had herkend, dit is Chewbacca die zingt onder de douche).

v = 1 ja / neen	v = 2 ja / neen
v = 3 ja / neen	v = 4 ja / neen

Wat zal het programma printen voor $v = 2$?

Wat zal het programma printen voor $v = 4$?

HOOFDSTUK 05

Herhalingen

Begrensde en voorwaardelijke herhalingen

BELANGRIJKE BEGRIPPEN

- for en range
- bereik
- geneste lussen
- patronen
- while

NA DIT HOOFDSTUK KUN JE

- Ik kan een begrensde herhaling schrijven en range() correct gebruiken.
- Ik kan tellers en accumulatoren opbouwen en traceren.
- Ik kan geneste lussen en uitvoerpatronen analyseren.
- Ik kan een while-lus veilig laten stoppen en passende grensgevallen testen.

Begrensde herhaling

Soms zijn er patronen in een programma, bijvoorbeeld in onderstaande code:

Code 5.1 – happy birthday

```
print("In 2024 ben ik 16 jaar.")
print("In 2025 ben ik 17 jaar.")
print("In 2026 ben ik 18 jaar.")
```

Hier wordt in essentie drie keer hetzelfde weergegeven, maar telkens met andere getallen. Je voelt wel aan dat het niet efficiënt is om dezelfde print()-instructie voortdurend te kopiëren en te plakken. Er bestaan twee controlestructuren om dit beter te implementeren. We beginnen met een **begrensde herhaling**, ofwel een for-lus.

Gebruik een begrensde herhaling of een for-lus wanneer je vooraf exact weet hoeveel keer een instructie moet worden herhaald.

Het voorbeeld uit code 5.1 kan eenvoudiger geschreven worden als:

```
for i in range(3):
    print("In", 2024 + i, "ben ik", 16 + i, "jaar.")
```

- De allereerste regel for i in range(3) kan je lezen als: "herhaal drie keer", of in meer detail "voor elke i van 0 tot (vóór) 3".
- De variabele i (afkorting van iterator) neemt dus achtereenvolgend de drie waarden 0, 1 en 2 aan.
- Hierdoor wordt 2024 + i achtereenvolgend 2024, 2025 en 2026, en analoog voor 16 + i.
- Merk op dat de print() instructie geïndenteerd werd!

Er zijn nog twee kleine varianten van de for-lus, namelijk:

- for i in range(1, 4)
 - Hierbij begint de variabele i vanaf 1 tot (vóór) 4. 1 als startgetal en 4 als eindgetal.
- for i in range(1, 4, 2)
 - De variabele i begint vanaf 1 tot (vóór) 4 maar maakt sprongen van 2 groot (=interval).

Code 5.2 – bereik of range

```
startjaar = int(input("Geef het startjaar in: "))
startleeftijd = int(input("Geef de startleeftijd in: "))
for i in range(3):
    print("In", startjaar + i, "ben ik", startleeftijd + i, "jaar.")
```

vorm	waarden van i
range(3)	
range(1, 4)	
range(1, 6, 2)	
range(5, 0, -1)	

Code 5.3 – Accumulator

Een belangrijke techniek bij het toepassen van een herhaling, is het continu aanpassen van variabelen. Stel dat je een programma wil schrijven dat de som van de eerste 10 volkomen kwadraten uitrekent:

```
som = 0
for i in range(10):
    som = som + i ** 2
print(som)
```

i	oude som	i ** 2	nieuwe som
0	0	0	0
1	0	1	1
2	1	4	5
3	5	9	14
...	...	...	...

Code 5.4 - Geneste herhaling

Net zoals je de selectiestructuur kon nesten, kan dit ook met herhalingen. Hieronder vind je een voorbeeld van een **geneste herhaling**. Let op: indents zijn van groot belang!

```
for x in range(2):
    for y in range(3):
        print(x + y)
```

Oefening 5.1 · range() voorspellen

Voorspel de uitvoer van volgende codefragmenten:

for i in range(4): print(i**2)	for i in range(1, 4): print(i**2)	for i in range(1, 4, 2): print(i**2)

Oefening 5.2 · Iteratorwaarden omvormen

Gebruik telkens `range(3)` en pas alleen de print-uitdrukking aan om de gevraagde uitvoer te genereren.

3	1	3
4	3	2
5	5	1
<code>for i in range(3):</code>	<code>for i in range(3):</code>	<code>for i in range(3):</code>

Oefening 5.3 · Lussen traceren

Welke getallen worden achtereenvolgend weergegeven bij het uitvoeren van volgende codefragmenten?

```
# Fragment 1
for i in range(2, 9, 2):
    print(i)
```

```
# Fragment 2
for i in range(0, 7, -1):
    print(i)
```

```
# Fragment 3
for cijfer in range(10, 0, -2):
    print(cijfer)
```

```
# Fragment 4
for i in range(0, 5):
    print(i)
```

```
# Fragment 5
teller = 0
for i in range(1, 4, 1):
    teller = teller + i
print(teller)
```

```
# Fragment 6
for a in range(1, 3):
    som = 0
    for b in range(3, 5):
        som = som + a + b
    print(som)
```

Oefening 5.4 · Vierkantwortels in stappen van vijf

Schrijf een programma dat de positieve vierkantwortels van 0, 5, 10, ... tot en met 100 toont.
Importeer math en kies een passende range().

Oefening 5.5 · Machtsverheffing als herhaald product

Vertrek van een programma dat $n * x$ als herhaalde som berekent.
Pas initialisatie en update aan zodat $x ** n$ als herhaald product wordt berekend.

```
n = int(input("n = "))
x = int(input("x = "))

product = 0
for i in range(n):
    product = product + x
print(product)
```

Oefening 5.6 · Drietallen met som 100

Onderstaand programma drukt alle verzamelingen van drie gehele getallen groter dan nul af waarvan de som gelijk is aan 100. De drie getallen van elke verzameling worden in oplopende volgorde afgedrukt.

Zo wordt het drietal 20 | 30 | 50 afgedrukt, maar nooit 50 | 20 | 30.

```
for i in range(1, 101):
    for j in range(1, 101):
        for k in range(1, 101):
            if i < j and j < k and i + j + k == 100:
                print(i, "|", j, "|", k)
```

Bron: be-OI 2024

a) Welk drietal wordt de eerste keer afgedrukt?

b) Wat is het tiende afgedrukt drietal?

c) Wat is het laatste afgedrukte drietal.

DIGITALE OEFENINGEN

Opstellen herhalingen

Open jouw oefenomgeving en los volgende oefening op:

- Som van Tien Getallen
- Bart Simpson
- Lancering
- Tafels
- Alle Tafels
- Delers
- Som van de Delers
- Volkomen Kwadraat
- Priemgetal
- Priemgetallen
- Griekse Vierkanten

Patronen tekenen

Om de begrensde herhaling echt grondig te beheersen moet je goed leren werken met de iterator `i`. Patronen tekenen is daarbij een zeer goede oefening. Vooraleer je aan de oefeningen begint is het nuttig om op te merken dat de instructie `print("A" * 4)` resulteert in de uitvoer "AAAA". Je kan deze concatenatie eenvoudig uitbreiden: `print("A" * 4 + "B" * 2)` resulteert in "AAAABB".

Oefening 5.7 · Een groeiend patroon tekenen

Teken de uitvoer van de lus.

```
for i in range(5):  
    print("X" * (i + 1))
```

Oefening 5.8 · Uitlijnen met spaties

Teken de uitvoer. Gebruik echte spaties in de tekenzone.

```
for i in range(5):  
    print(" " * (4 - i) + "X" * (i + 1))
```

Oefening 5.9 · Een krimpend patroon schrijven

Schrijf code die op de eerste regel zeven X-tekenen toont en daarna elke regel een X minder tot er een overblijft.

```
XXXXXXX  
XXXXXX  
XXXXX  
XXXX  
XXX  
XX  
X
```


Oefening 5.10 · Een hol patroon opbouwen

Analyseer een patroon met een afwijkende eerste regel en daarna twee X-tekens met veranderende tussenruimte. Vul een tabel met i, buitenste spaties en binnenste spaties en schrijf de lus met een selectie.

- · stelt een spatie voor

```
....X
...X.X
..X...X
.X.....X
X.....X
```

i	spaties 1	X	spaties 2	X
0		1	-	-
1		1		1
2		1		1
3		1		1
4		1		1

```
for i in range(...):
    aantal_1 = ...
    aantal_2 = ...
    if i == ...:
        ...
    else:
        ...
```

Oefening 5.11 · Afwijkende eerste en laatste regel

Benoem welke regels van het patroon afwijken. Ontwerp variabelen voor de verschillende aantallen spaties en schrijf een lus met passende selecties.

-- stelt een spatie voor

```
X.....X
XX.....X
X.X.....X
X..X.....X
X...X.....X
X....X.....X
X.....XX
X.....X
```

i					
0	1		5 spaties		1
1					
2					
3					
4					
5					
6	1		5 spaties		1

```
for i in range(...):
    ...
```

Oefening 5.12 · Afwisselende regels met modulo

Lees n in en toon n regels. Op even rijnummers verschijnt een volledige rij X-tekens; op oneven rijnummers alleen een X aan de rechterkant. Gebruik $i \% 2$.

Bij $n = 4$

XXXX

__X

XXXX

__X

Bij $n = 5$

XXXXX

__X

XXXXX

__X

XXXXX

Random-functie

Soms wil je dat een programma niet altijd exact hetzelfde doet, bijvoorbeeld bij een dobbelsteenworp of een willekeurige oefenvraag. Python bewaart de opdrachten hiervoor in de module `random`. Net zoals bij `math` moet je deze module eerst importeren.

DEFINITIE

`random.randint(a, b)`

Maakt één willekeurig geheel getal van `a` tot en met `b`. Beide grenzen zijn inbegrepen:
`random.randint(1, 6)` kan 1, 2, 3, 4, 5 of 6 opleveren.

Wanneer `random.randint()` in een `for`-lus staat, maakt Python bij elke herhaling een nieuwe keuze. Dit programma simuleert vijf worpen met één dobbelsteen:

Voorbeeld – Vijf willekeurige dobbelsteenworpen

```
import random

for i in range(5):
    worp = random.randint(1, 6)
    print("Worp", i + 1, "=", worp)
```

De exacte uitvoer kun je vooraf niet voorspellen. Je kunt wel controleren wat altijd waar moet zijn: er verschijnen vijf regels, de nummering loopt van 1 tot en met 5 en elke worp ligt tussen 1 en 6. Een andere uitvoer bij een volgende uitvoering is dus geen fout.

DIGITALE OEFENINGEN

Random functie

Open jouw oefenomgeving en los volgende oefening op:

- Willekeurige Maaltafel
- Test Jouw Maaltafels
- Simulatie Dobbelsteenworpen

Introductie lijsten

Tot nu toe bewaarde een variabele meestal één waarde. Soms horen verschillende waarden bij elkaar, zoals temperaturen van één week of de drie waarden van een RGB-kleur. In Python bewaar je zulke waarden samen in een **lijst**. De elementen staan tussen vierkante haken en zijn door komma's gescheiden.

DEFINITIE

Lijst

Een geordende verzameling waarden. Elk element heeft een index of rangnummer. Dit geeft de plaats van de waarde binnen de lijst aan. Bij Python begint bij index 0; de laatste geldige index is `len(lijst) - 1`.

In `temperaturen = [17, 19, 22]` staat 17 op index 0, 19 op index 1 en 22 op index 2. Met `temperaturen[0]` vraag je het eerste element op. Een index buiten het geldige bereik veroorzaakt een `IndexError`.

Voorbeeld – Een item opvragen uit een lijst

```
temperaturen = [17, 19, 22]
print("Eerste meting:", temperaturen[0])
```

Een lijst kan tijdens het programma groeien. Met `.append()` voeg je één element achteraan toe. De functie `len()` geeft het aantal elementen. Wil je alle waarden gebruiken, dan itereer je het duidelijkst rechtstreeks over de lijst.

Voorbeeld – Een lijst opbouwen en doorlopen

```
temperaturen = [17, 19, 22]
print("Eerste meting:", temperaturen[0])

temperaturen.append(24)
print("Aantal metingen:", len(temperaturen))

for temperatuur in temperaturen:
    print(temperatuur)
```

Dit programma toont eerst 17, daarna 4 en vervolgens de vier temperaturen in dezelfde volgorde als in de lijst.

OPGELET

Lengte en laatste index zijn niet hetzelfde

Na `kleuren = ["rood", "groen"]` en `kleuren.append("blauw")` is de lengte 3. Welke waarde staat op index 0 en wat is de hoogste geldige index? Verklaar het verschil tussen beide uitkomsten.

DIGITALE OEFENINGEN

Intro Lijsten en Append

Open jouw oefenomgeving en los volgende oefening op:

- Verkoopsaantallen
- RGB

Uitbreiding: rij van Fibonacci

De rij van Fibonacci is een getallenrij met een eenvoudig patroon: elk nieuw getal is de som van de twee vorige getallen. In deze cursus gebruiken we volgende afspraak:

AFSPRAAK

$$u_0 = 1 \text{ en } u_1 = 1$$

Voor n vanaf 2 geldt $u_n = u_{n-1} + u_{n-2}$. De rij begint dus met 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Om één gevraagde waarde te berekenen, hoef je niet de volledige rij te bewaren. vorige en huidige bewaren de twee laatst bekende waarden. Eerst bereken je volgende; daarna schuif je de waarden één plaats door. De volgorde is belangrijk: bereken de som voordat je een oude waarde overschrijft.

Te berekenen rangnummer	Optelling	Nieuwe huidige waarde
2	1 + 1	2
3	1 + 2	3
4	2 + 3	5
5	3 + 5	8

Voorbeeld – De waarde op rangnummer 5 berekenen

```
vorige = 1
huidige = 1

for rangnummer in range(2, 6):
    volgende = vorige + huidige
    vorige = huidige
    huidige = volgende

print(huidige)
```

De uitvoer is 8. De waarden voor rangnummer 0 en 1 zijn bij de start al bekend; daarom begint de lus bij 2. In de oefeningen veralgemeen je dit algoritme voor een ingevoerd rangnummer n met $n \geq 0$.

DIGITALE OEFENINGEN

Random functie

Open jouw oefenomgeving en los volgende oefening op:

- Rij van Fibonacci

Itereren over een string

Een string is een rij tekens. Met een for-lus doorloop je die tekens één voor één. Je hebt daarvoor geen teller of index nodig: bij elke herhaling krijgt de lusvariabele het volgende teken uit de string.

DEFINITIE

Itereren over een string

Elk teken van de string achtereenvolgens verwerken.

Voorbeeld – Elk teken afzonderlijk weergeven

```
woord = "Python"

for teken in woord:
    print(teken)
```

De lus wordt zes keer uitgevoerd, want `len(woord)` is 6. Eerst is teken gelijk aan "P", daarna aan "y", enzovoort.

Soms bouw je tijdens de lus ook een nieuwe string op. Start dan met een lege string en pas die bij elke herhaling aan.

Voorbeeld – Een nieuwe string opbouwen

```
woord = "kat"
nieuw = ""

for teken in woord:
    nieuw = nieuw + "[" + teken + "]"

print(nieuw)
```

De uitvoer is `[k][a][t]`. De variabele `nieuw` bewaart telkens het tussentijdse resultaat. Elk teken heeft daarnaast een index. Net als bij een lijst is de eerste index 0 en de laatste geldige index `len(woord) - 1`. Gebruik een index alleen wanneer de positie nodig is of wanneer je naburige tekens wilt vergelijken.

Voorbeeld – Tekens met hun index weergeven

```
woord = "Python"

for i in range(len(woord)):
    print(i, woord[i])
```

Hier doorloopt `i` de waarden 0 tot en met 5. De uitdrukking `woord[i]` kiest het teken op die positie. Een index buiten het geldige bereik veroorzaakt een `IndexError`.

DIGITALE OEFENINGEN**Random functie**

Open jouw oefenomgeving en los volgende oefening op:

- Woord omdraaien
- Palindroom
- P-taal
- Letters Tellen
- Isogram
- Game Rank
- Game Rank Deel 2
- Dubbele Klinkers

Voorwaardelijke herhaling

Gebruik een while-lus wanneer het aantal herhalingen niet vooraf vastligt, maar wel een duidelijke stopvoorwaarde bestaat. Je weet dus op voorhand niet hoeveel keer onze lus herhaald wordt! Zorg dat de toestand in de lus verandert zodat de voorwaarde uiteindelijk False wordt.

We leggen dit nieuwe programmeerconcept uit aan de hand van een gedachtenexperiment waarbij we papier vouwen tot de maan. De dikte van een blad papier is ongeveer 0,01 cm en de afstand van de aarde tot de maan ongeveer 384400 km. Hoeveel keer moet je het blad papier dubbelvouwen zodat het tot de maan geraakt?

Na één keer vouwen is het blad 0,02 cm dik, na twee keer 0,04 cm, na drie keer 0,08 cm, enzovoort. De dikte van het blad verdubbelt dus telkens en dit proces moet een aantal keer worden herhaald. Een naïve manier om dit in Python te implementeren gaat als volgt:

Code 5.5 - Papier vouwen tot de maan

```
dikte = 0.01
for i in range(????):
    dikte = dikte * 2
    print(dikte)
```

Maar het is onduidelijk wat op de plaats van de vraagtekens ???? hoort te staan. We weten op voorhand immers niet hoeveel keer het blad gevouwen moet worden. We weten wel dat we moeten blijven vouwen zolang (while) de dikte kleiner is dan de afstand tussen de aarde en de maan. Met andere woorden:

```
dikte = 0.01
while dikte < 384400 * 10 ** 5:
    dikte = dikte * 2
    print(dikte)
```

De vermenigvuldiging met $10 ** 5$ werd toegevoegd omdat de dikte in cm wordt uitgedrukt, terwijl de afstand tussen de aarde en de maan in km wordt gegeven.

Dit programma zal de achtereenvolgende diktes op het scherm weergeven, maar pas na alle regels te tellen weet je hoeveel keer je het blad moet vouwen. De iterator `i` uit de begrensde herhaling ontbreekt hier, al kan deze manueel toegevoegd worden:

```
dikte = 0.01
afstand_cm = 384400 * 100000
aantal = 0

while dikte < afstand_cm:
    dikte = dikte * 2
    aantal = aantal + 1

print(aantal)
```

Code 5.6 – Kies voor de kortste methode

Begrensde herhaling	Voorwaardelijke herhaling
<pre>som = 0 for i in range(10): som = som + i**2 print(som)</pre>	<pre>som = 0 i = 0 while i < 10: som = som + i**2 i = i + 1 print(som)</pre>

Een while-lus is langer omdat je de iterator zelf aanmaakt en aanpast.
Gebruik daarom bij voorkeur een for-lus wanneer je het aantal herhalingen vooraf kent.

DIGITALE OEFENINGEN**Opstellen van een voorwaardelijke herhaling**

Open jouw oefenomgeving en los volgende oefening op:

- Oneindige Lus
- Papier Vouwen naar de Maan
- 200 Gram Suiker

Oefening 5.13 · for omzetten naar while

Vorm onderstaande programma's met een for-lus om naar een while-lus.

```
for i in range(1, 10, 2):  
    print(10 - i)
```

```
product = 1  
for i in range(1, 5):  
    product = product * i  
print(product)
```

Oefening 5.14 · Een oneindige lus analyseren en herstellen

Voer dit fragment niet uit.

Leg uit waarom de lus niet stopt en debug nadien de code.

```
i = 10  
while i > 0:  
    print(i)
```

Oefening 5.15 · Stopvoorwaarden onderzoeken

Wat verschijnt op het scherm na het uitvoeren van onderstaande codefragmenten?

```
# Fragment 1
uitkomst = 100
while uitkomst != 16:
    uitkomst *= 0.4
print(uitkomst)
```

```
# Fragment 2
uitkomst = 100
while uitkomst < 120:
    uitkomst *= 1.2
print(uitkomst)
```

```
# Fragment 3
uitkomst = 100
while uitkomst <= 120:
    uitkomst *= 1.2
print(uitkomst)
```

NIEUW

teller += 1 is een verkorte schrijfwijze voor teller = teller + 1.
Beide vormen passen dezelfde variabele aan.

DIGITALE OEFENINGEN

Opstellen herhalingen

Open jouw oefenomgeving en los volgende oefening op:

- Kudde Antilopen
- Stuiterende Bal
- R0-Waarde
- Sommen Maken
- Klasresultaten en Gemiddelden
- Ontbinden in Priemfactoren
- Spaarbank Deel 2

Oefening 5.16 · De laatste uitvoer bepalen

Bepaal voor drie lussen wat de **allerlaatste afgedrukte** waarde is.

```
# Fragment 1
eindnummer = 4
teller = 2
while teller <= eindnummer:
    teller = teller + 1
    print(teller)
```

```
# Fragment 2
eindnummer = 4
teller = 2
while teller <= eindnummer:
    print(teller)
    teller = teller + 1
```

```
# Fragment 3
eindnummer = 4
teller = 2
while teller <= eindnummer:
    teller += 1
    if teller == 4:
        teller = teller + eindnummer
    print(teller)
```

DIGITALE OEFENINGEN

Spelen met Herhalingen

Open jouw oefenomgeving en los volgende oefening op:

- Hoger Lager
- Hoger Lager meer met Liegen
- Blackjack
- Dubbele Zessen
- Rock Paper Scissors

Overzicht leerdoelen

Computationeel denken

- Ik ontleed een probleem in invoer, verwerking en uitvoer.
- Ik ontwerp een algoritme voordat ik code schrijf.
- Ik test normale gevallen, grensgevallen en foutieve invoer.

Python-basis

- Ik gebruik duidelijke variabelen en passende datatypes.
- Ik lees invoer, verwerk waarden en maak duidelijke uitvoer.
- Ik herken syntax-, runtime- en logische fouten.

Controlestructuren

- Ik schrijf sequenties in de juiste volgorde.
- Ik bouw een- en meerzijdige selecties.
- Ik kies tussen for en while en zorg dat lussen stoppen.

Begrippenlijst

<code>algoritme</code>	Een eindige, ondubbelzinnige reeks stappen.
<code>variabele</code>	Een naam die verwijst naar een waarde in het geheugen.
<code>datatype</code>	De soort waarde, bijvoorbeeld int, float, str of bool.
<code>initialiseren</code>	Een variabele voor het eerst een beginwaarde geven.
<code>syntaxfout</code>	Code die niet aan de taalregels van Python voldoet.
<code>runtime-fout</code>	Een fout die tijdens het uitvoeren ontstaat.
<code>logische fout</code>	Code die uitvoert maar een verkeerd resultaat oplevert.
<code>gehele deling</code>	De operator <code>//</code> geeft het gehele quotiënt.
<code>modulo</code>	De operator <code>%</code> geeft de rest van een deling.
<code>selectie</code>	Een controlestructuur die code op basis van een voorwaarde kiest.
<code>iterator</code>	De variabele die bij een for-lus opeenvolgende waarden krijgt.
<code>accumulator</code>	Een variabele waarin een resultaat stap voor stap wordt opgebouwd.
<code>begrensde herhaling</code>	Een for-lus met een vooraf bepaald aantal stappen.
<code>voorwaardelijke herhaling</code>	Een while-lus die doorgaat zolang een voorwaarde waar is.
<code>debuggen</code>	De oorzaak van een fout zoeken, verbeteren en opnieuw testen.

Bronnen en verantwoording

- Arnaud Bodin. Python in High School: Algorithms and Mathematics. Springer, 2020.
- Leen Brouns. Nascholing Algoritmen en Programmeren, didactische wenken. Universiteit Gent, 2023.
- Paul J. Deitel en Harvey M. Deitel. Java: How to Program. Pearson Education, 2012.
- Svein Linge. Programming for Computations - Python. Springer, 2016.
- Vanderfaeillie D. en Wulgaert R.: Python in de Klas, online cursus Dodona, 2026
- be-OI-opgaven 2016, 2017, 2023 en 2024, met bronvermelding bij de betreffende oefening.
- Wulgaert R.: Project Delphi, 2026

Formularium

Begrip of statement	Beschrijving	Syntax	Let op
print()	Schrijft tekst of waarden naar de uitvoer.	<code>print("Hallo")</code>	De uitvoer moet vaak exact overeenkomen met de testcases. Controleer hoofdletters, spaties en leestekens.
Variabele en =	Bewaart een waarde onder een naam.	<code>naam = "Ada"</code>	Gebruik <code>=</code> om een waarde toe te kennen. Gebruik <code>==</code> om twee waarden te vergelijken.
input()	Vraagt invoer aan de gebruiker.	<code>naam = input()</code>	Het resultaat van <code>input()</code> is altijd tekst: een <code>str</code> .
int()	Zet een geschikte waarde om naar een geheel getal.	<code>int("14")</code>	Bij een float verdwijnen de decimalen naar nul toe: <code>int(-7.9)</code> is <code>-7</code> . Er wordt niet afgerond.
float()	Zet een geschikte waarde om naar een kommagetal.	<code>float("3.29")</code>	Gebruik een punt in een getal dat Python als kommagetal moet lezen.
str()	Zet een waarde om naar tekst.	<code>str(score)</code>	Nodig wanneer je een getal met tekst wilt samenplakken met <code>+</code> .
Booleaanse waarden	True en False zijn de twee waarden van het type <code>bool</code> . Vergelijkingen leveren zo'n waarde op.	<code>klaar = True</code> <code>groot = getal > 10</code>	Schrijf True en False met een hoofdletter en zonder aanhalingstekens.
Komma's in print()	Combineert tekst en waarden in één uitvoeropdracht.	<code>print("Score:", score)</code>	Python voegt standaard een spatie tussen de argumenten toe.
String-concatenatie	Plakt tekststukken samen.	<code>"Score: " + str(score)</code>	Beide kanten van <code>+</code> moeten strings zijn. Zet getallen eerst om met <code>str()</code> .
Rekenoperatoren	Voeren gewone berekeningen uit.	<code>+</code> <code>-</code> <code>*</code> <code>/</code>	<code>/</code> levert ook bij twee gehele getallen een float op. Delen door nul veroorzaakt een fout.

Begrip of statement	Beschrijving	Syntax	Let op
Macht **	Berekent een macht.	zijde ** 2	Links staat het grondtal, rechts de exponent. Gebruik haakjes wanneer de rekenvolgorde onduidelijk is.
Gehele deling //	Berekent het quotiënt en rondt dit naar beneden af.	17 // 5 # 3 -17 // 5 # -4	Bij niet-negatieve waarden is dit het quotiënt zonder rest. De deler mag niet 0 zijn.
Modulo / rest %	Geeft de rest die bij de gehele deling hoort.	17 % 5 # 2 -17 % 5 # 3	Negatieve waarden kunnen verrassen. De deler mag niet 0 zijn.
round()	Rondt af op hele eenheden of op een opgegeven aantal decimalen.	round(x) round(x, 2)	Precies-halverwegegevallen volgen Python's afrondingsregel: round(2.5) is 2 en round(3.5) is 4.
min() en max()	Kiezen de kleinste of grootste vergelijkbare waarde.	min(a, b) max(a, b)	De waarden moeten met elkaar vergelijkbaar zijn.
import	Laadt een module met extra functies en constanten.	import math import random	Zet imports bij voorkeur bovenaan het programma en schrijf de modulenaam voor de functie.
math.sqrt()	Berekent de niet-negatieve vierkantswortel.	math.sqrt(x)	Vereist import math en $x \geq 0$. Een negatieve invoer veroorzaakt ValueError.
math.pi	Geeft Python's benadering van het getal pi.	math.pi	Vereist import math. Gebruik dit liever dan zelf 3.14 te typen.
math.ceil() / floor()	Ronden respectievelijk naar boven en naar beneden af op een geheel getal.	math.ceil(x) math.floor(x)	Vereist import math. ceil kiest het kleinste gehele getal $\geq x$; floor het grootste gehele getal $\leq x$.
random.randint()	Maakt een willekeurig geheel getal tussen twee grenzen.	random.randint(1, 6)	Beide grenzen zijn inbegrepen. Elke oproep maakt een nieuwe keuze.
if	Voert een blok uit als de voorwaarde True is.	if voorwaarde: opdracht	Het dubbele punt is verplicht. Het blok eronder springt in.

Begrip of statement	Beschrijving	Syntax	Let op
elif	Test een volgende voorwaarde als eerdere voorwaarden False waren.	elif voorwaarde: opdracht	Je mag meerdere elif-blokken in één if/elif-keten gebruiken.
else	Vangt alle overige gevallen in een if/elif-keten op.	else: opdracht	Per keten is er maximaal één else, altijd als laatste en zonder voorwaarde.
Vergelijkingen	Vergelijken twee waarden en leveren True of False op.	== != < <= > >=	== vergelijkt; = kent een waarde toe.
and	Combineert voorwaarden; beide moeten True zijn.	a > 0 and b > 0	De volledige combinatie is alleen True als beide delen True zijn.
or	Combineert voorwaarden; minstens één moet True zijn.	a > 0 or b > 0	De volledige combinatie is True zodra minstens één deel True is.
not	Keert de Booleanwaarde van een voorwaarde om.	if not klaar: opdracht	not maakt True tot False en False tot True.
Dubbelepunt :	Sluit de kopregel van een blok af.	if ...: for ...: while ...:	Een ontbrekend dubbelepunt veroorzaakt SyntaxError.
Indent / inspringing	Toont welke opdrachten bij een keuze of lus horen.	if klaar: print("Start")	Gebruik per niveau consequent vier spaties en meng geen tabs en spaties. Een ander niveau kan de nesting veranderen.
for	Doorloopt elk element van een reeks.	for element in reeks: opdracht	Gebruik for ... in range(...) wanneer het aantal stappen vooraf bekend is.
range()	Maakt een getallenreeks waarvan de stopwaarde niet meetelt.	range(stop) range(start, stop)	range(5) bevat 0, 1, 2, 3 en 4. De stopwaarde zelf zit niet in de reeks.
range() met stap	Maakt een getallenreeks met een vaste positieve of negatieve stap.	range(2, 11, 2) range(5, 0, -1)	De stap mag niet 0 zijn. Het teken van de stap moet in de richting van de stopwaarde gaan.

Begrip of statement	Beschrijving	Syntax	Let op
Teller / accumulator	Bewaart een veranderend tussenresultaat in een lus.	<pre>totaal = 0 totaal = totaal + getal</pre>	Initialiseer de variabele vóór de lus en pas ze binnen de lus aan.
+= en *=	Verkorte vormen die een bestaande variabele aanpassen.	<pre>teller += 1 product *= factor</pre>	De variabele moet eerst bestaan. Gebruik de verkorte vorm alleen als de lange vorm duidelijk is.
Geneste herhaling	Een lus bevat een andere lus. De binnenste lus doorloopt opnieuw alle stappen bij elke buitenste stap.	<pre>for x in range(2): for y in range(3): print(x, y)</pre>	Let op de inspringing: de volledige binnenste lus hoort bij de buitenste lus.
while	Herhaalt een blok zolang de voorwaarde True is.	<pre>while voorwaarde: opdracht</pre>	Zorg dat de toestand in de lus naar een stopvoorwaarde evolueert.
Invoer tot stopwoord	Vraagt invoer tot de gebruiker het stopwoord geeft.	<pre>antwoord = input() while antwoord != "stop": print(antwoord) antwoord = input()</pre>	Initialiseer vóór de lus en vraag opnieuw als laatste stap van het lusblok.
while tot doel	Herhaalt tot een meetbare doelwaarde bereikt is.	<pre>while saldo < doel: saldo = saldo * 1.02</pre>	Controleer dat elke herhaling de waarde werkelijk dichterbij het doel brengt.
Lijstliteral []	Bewaart een geordende reeks waarden in één variabele.	<pre>waarden = [17, 19, 22]</pre>	Elementen staan tussen vierkante haken en zijn door komma's gescheiden.
Index	Kiest één element van een lijst of één teken van een string.	<pre>waarden[0] woord[2]</pre>	De eerste index is 0; de laatste geldige index is <code>len(reeks) - 1</code> . Buiten het bereik volgt <code>IndexError</code> .
.append()	Voegt één element achteraan een lijst toe.	<pre>waarden.append(24)</pre>	<code>append()</code> verandert de bestaande lijst en voegt precies één element toe.
len()	Geeft het aantal elementen of tekens in een reeks.	<pre>len(kleuren) len(woord)</pre>	De lengte is niet de laatste index. Bij een niet-lege reeks is de laatste index <code>len(reeks) - 1</code> .

Begrip of statement	Beschrijving	Syntax	Let op
Lijst itereren	Doorloopt de waarden van een lijst rechtstreeks.	<code>for kleur in kleuren: print(kleur)</code>	Gebruik een index alleen wanneer de positie deel uitmaakt van het probleem.
String itereren	Doorloopt de tekens van een string rechtstreeks.	<code>for teken in woord: print(teken)</code>	De lusvariabele bevat telkens één teken. Aanhalingstekens maken geen deel uit van de string.
String met index	Doorloopt posities wanneer de index nodig is.	<code>for i in range(len(woord)): print(i, woord[i])</code>	De indexen lopen van 0 tot en met <code>len(woord) - 1</code> .
String opbouwen	Bewaart stap voor stap een nieuwe string in een accumulator.	<code>nieuw = "" for teken in woord: nieuw = nieuw + teken</code>	Initialiseer de accumulator vóór de lus met de lege string.

